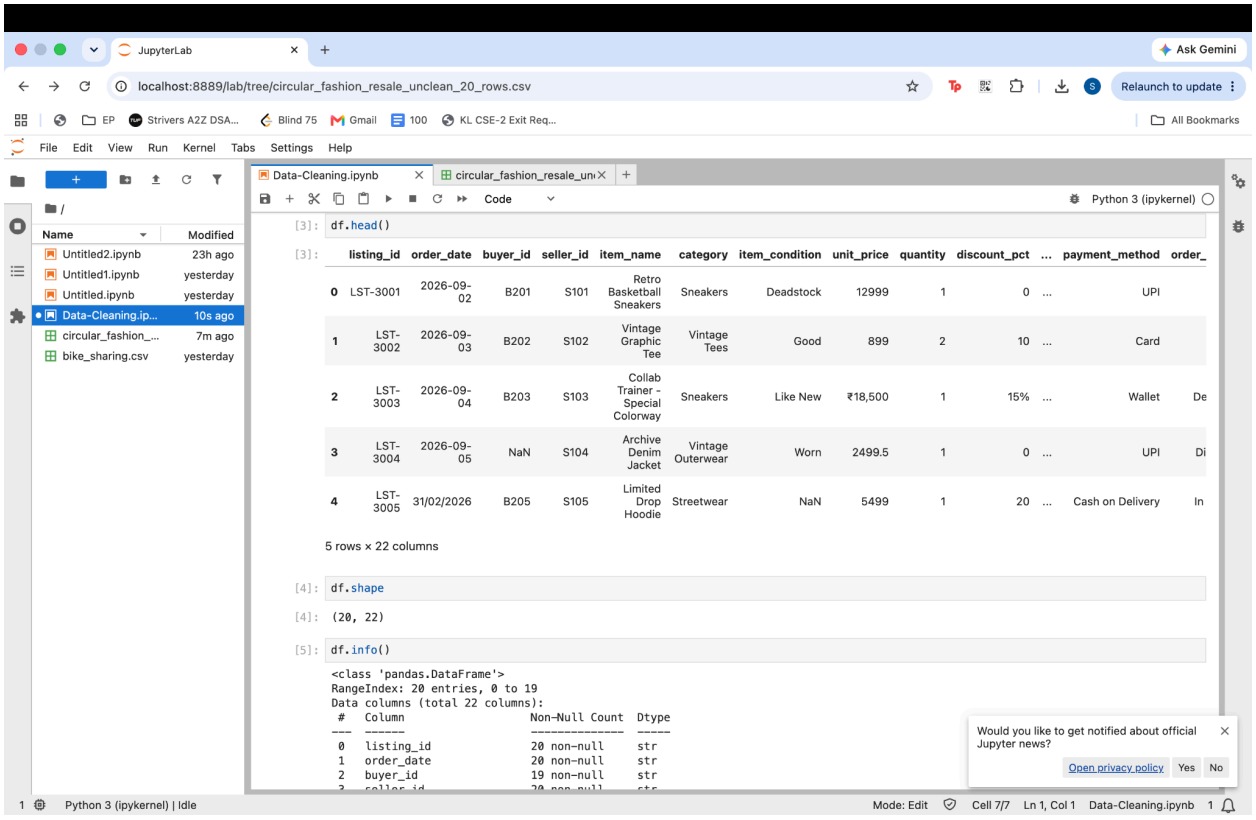
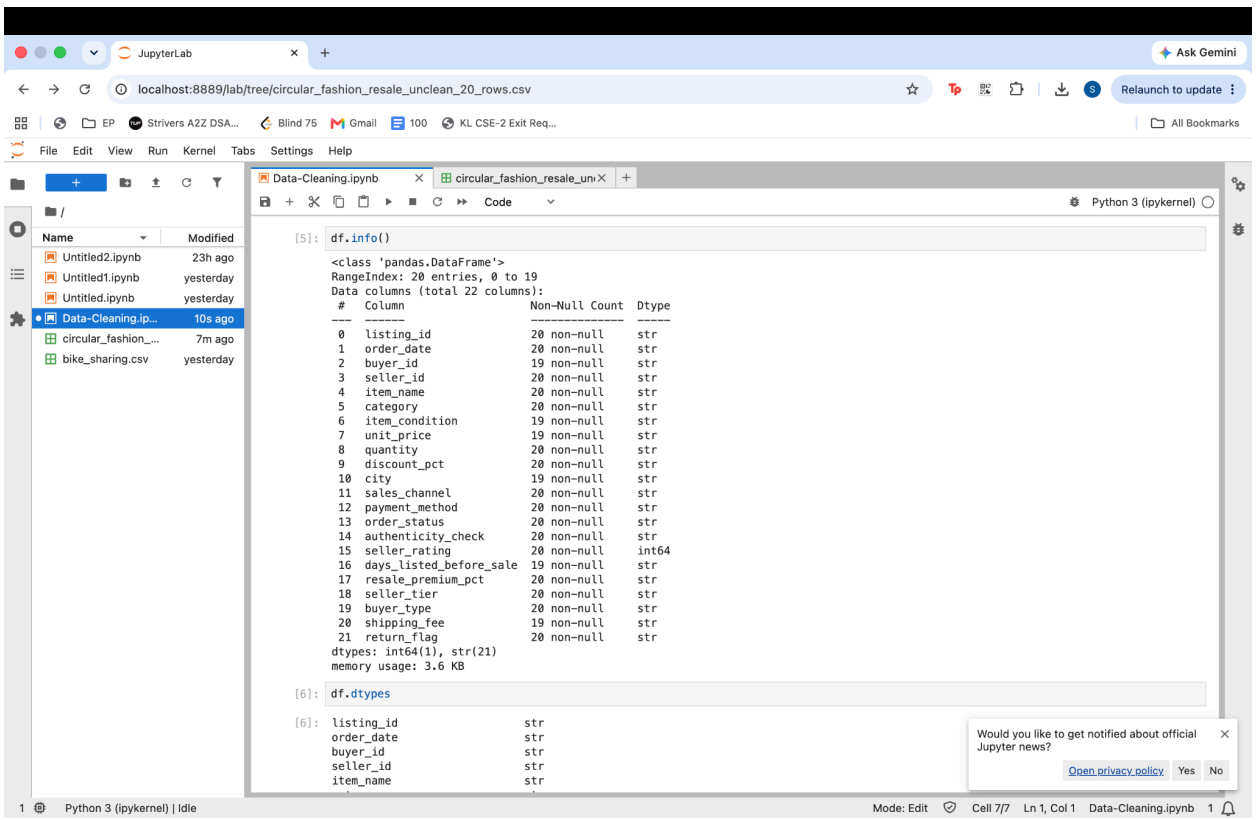
Data inspection:



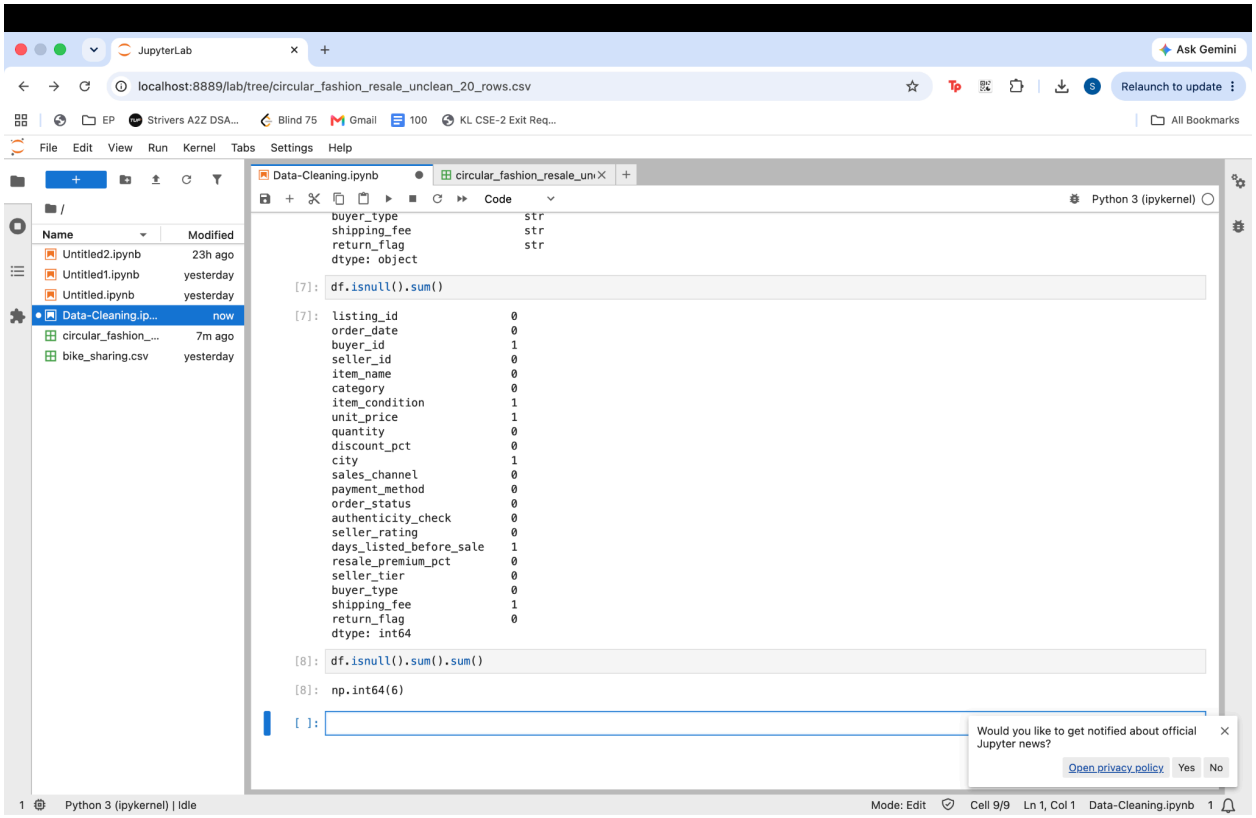

| | listing_id | order_date | buyer_id | seller_id | item_name | category | item_condition | unit_price | quantity | discount_pct | ... | payment_method | order_ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | LST-3001 | 2026-09-02 | B201 | S101 | Retro Basketball Sneakers | Sneakers | Deadstock | 12999 | 1 | 0 | ... | UPI | |
| 1 | LST-3002 | 2026-09-03 | B202 | S102 | Vintage Graphic Tee | Vintage Tees | Good | 899 | 2 | 10 | ... | Card | |
| 2 | LST-3003 | 2026-09-04 | B203 | S103 | Collab Trainer - Special Colorway | Sneakers | Like New | ₹18,500 | 1 | 15% | ... | Wallet | De |
| 3 | LST-3004 | 2026-09-05 | NaN | S104 | Archive Denim Jacket | Vintage Outerwear | Worn | 2499.5 | 1 | 0 | ... | UPI | Di |
| 4 | LST-3005 | 31/02/2026 | B205 | S105 | Limited Drop Hoodie | Streetwear | NaN | 5499 | 1 | 20 | ... | Cash on Delivery | In |

5 rows × 22 columns

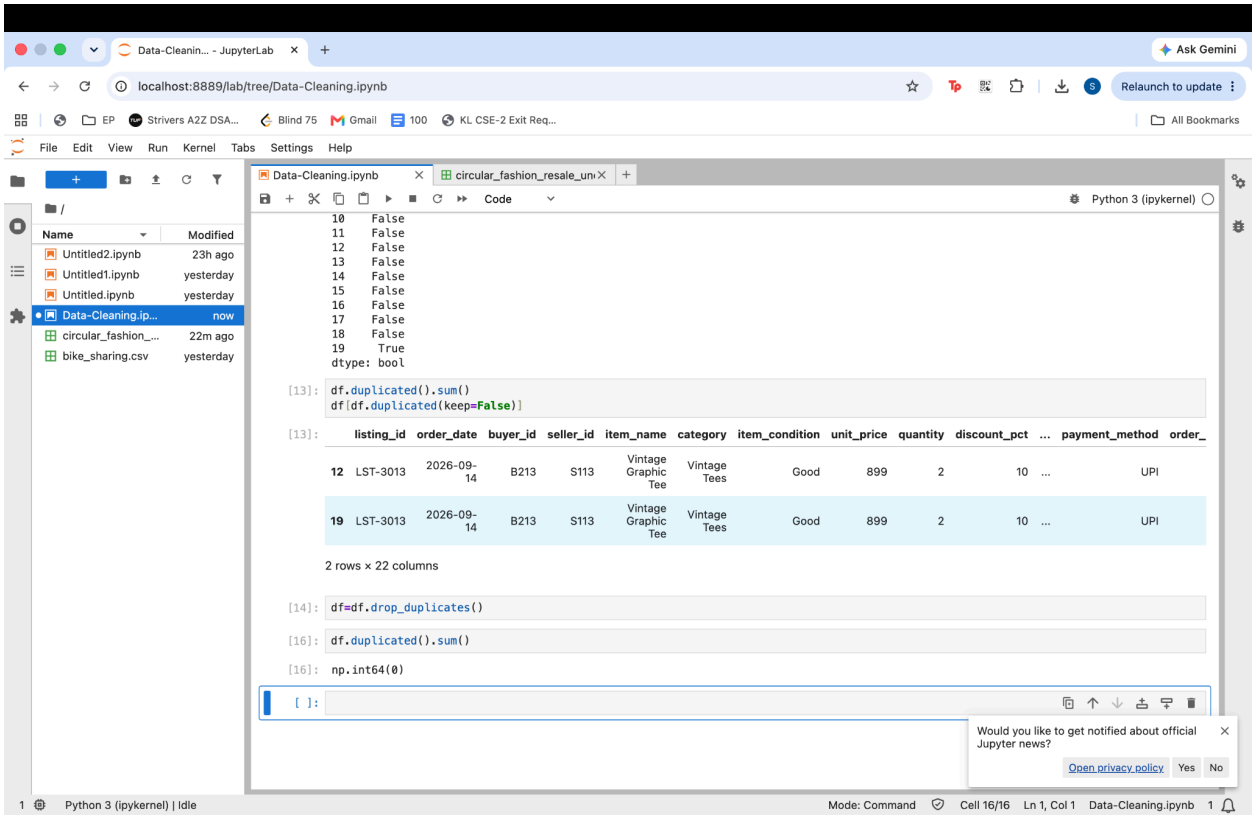
| # | Column | Non-Null Count | Dtype |
|---|---|---|---|
| 0 | listing_id | 20 non-null | str |
| 1 | order_date | 20 non-null | str |
| 2 | buyer_id | 19 non-null | str |



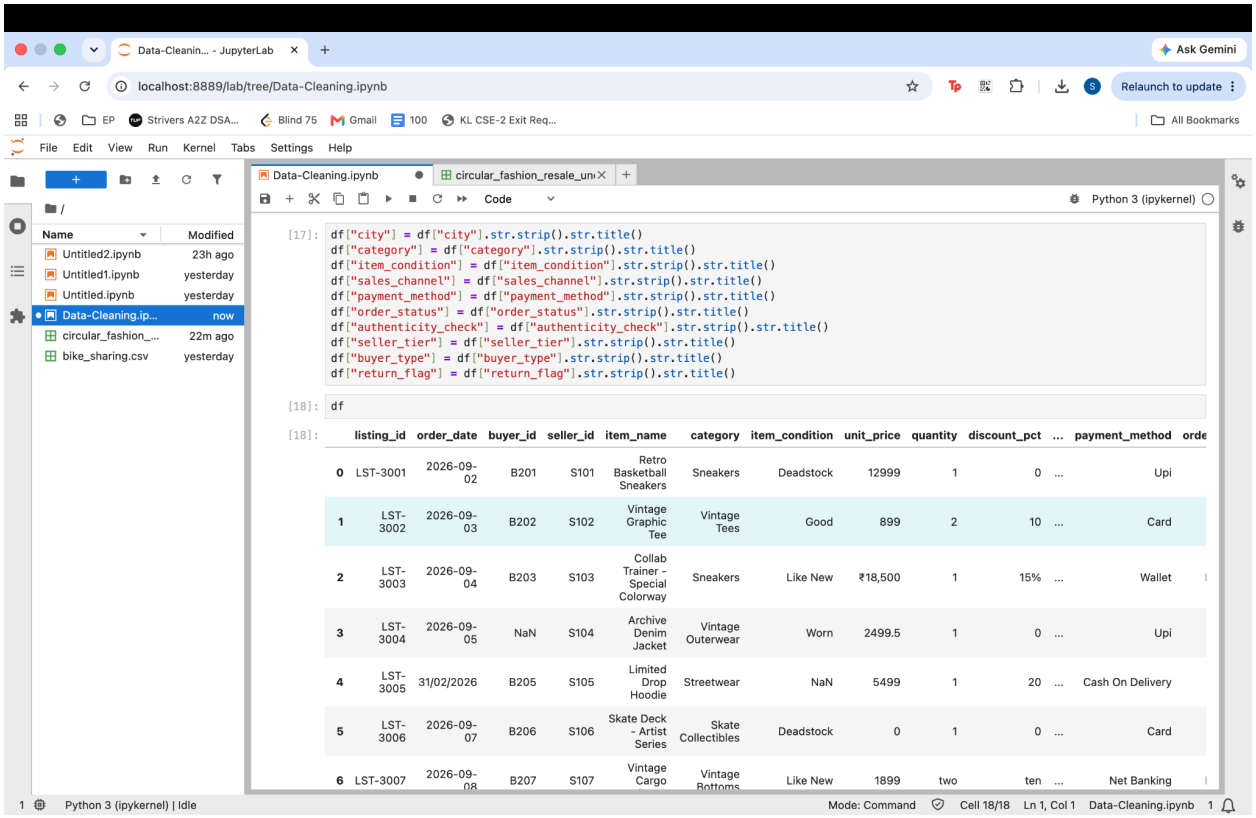
Missing values:



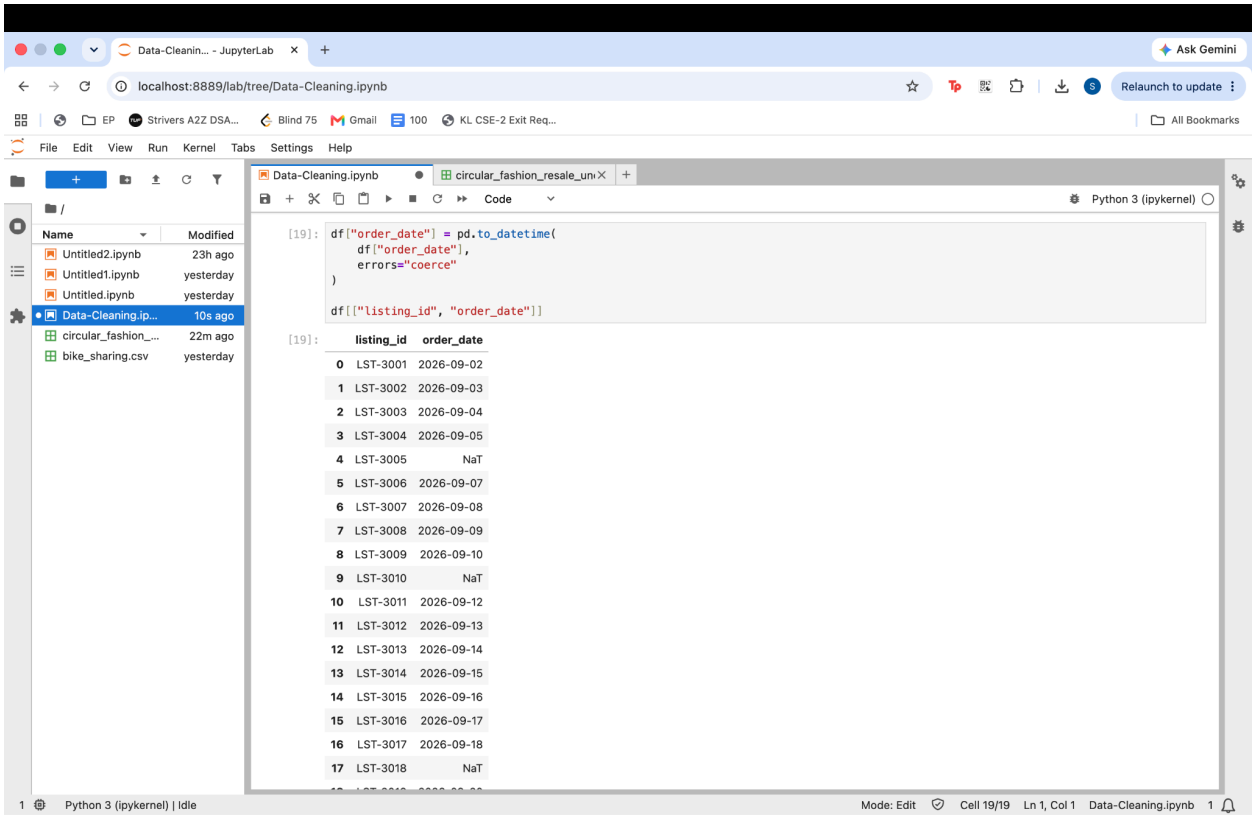

Finding duplicates and dropping duplicates:



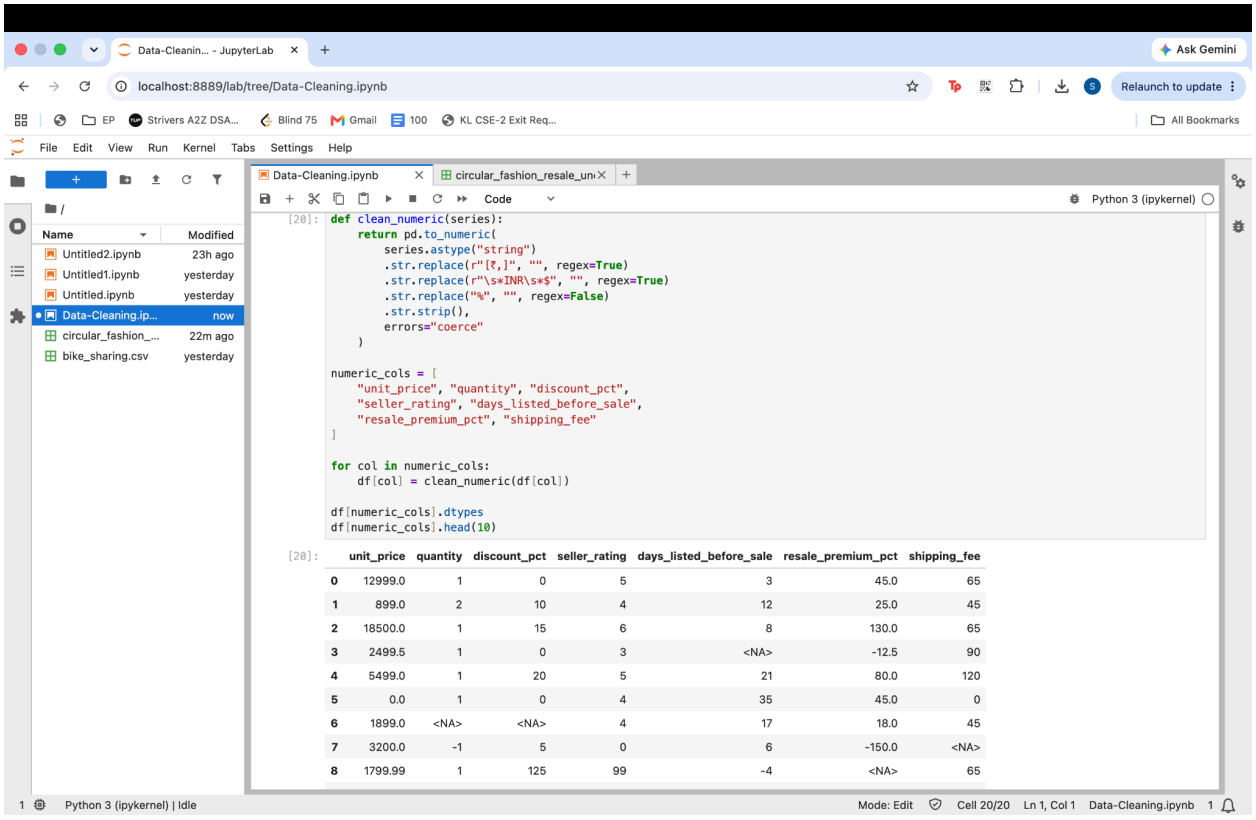
Standardizing text:




Converting to proper date time format:



Converting to numerics:



```
def clean_numeric(series):
    return pd.to_numeric(
        series.astype("string")
        .str.replace(r"[₹,]", "", regex=True)
        .str.replace(r"\s*INR\s*$", "", regex=True)
        .str.replace("%", "", regex=False)
        .str.strip(),
        errors="coerce"
    )

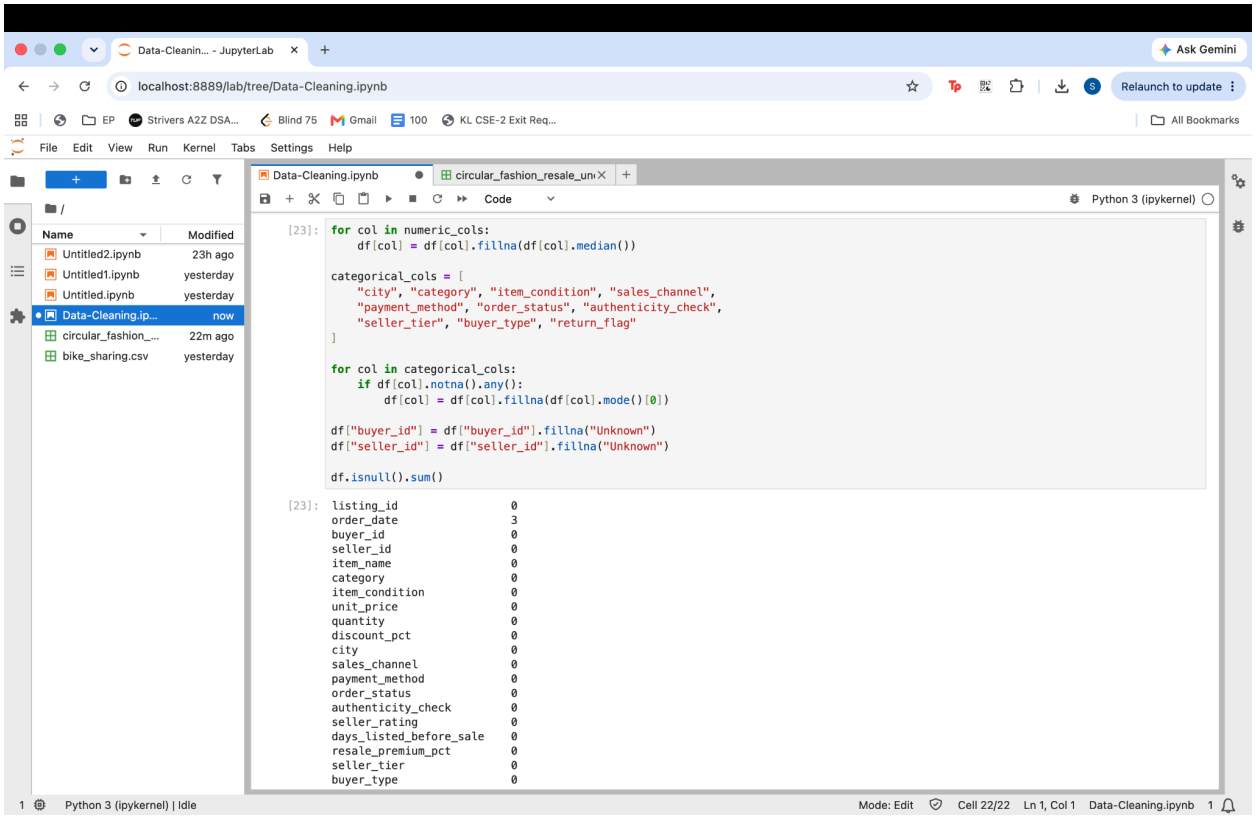
numeric_cols = [
    "unit_price", "quantity", "discount_pct",
    "seller_rating", "days_listed_before_sale",
    "resale_premium_pct", "shipping_fee"
]

for col in numeric_cols:
    df[col] = clean_numeric(df[col])

df[numeric_cols].dtypes
df[numeric_cols].head(10)
```

| | unit_price | quantity | discount_pct | seller_rating | days_listed_before_sale | resale_premium_pct | shipping_fee |
|---|---|---|---|---|---|---|---|
| 0 | 12999.0 | 1 | 0 | 5 | 3 | 45.0 | 65 |
| 1 | 899.0 | 2 | 10 | 4 | 12 | 25.0 | 45 |
| 2 | 18500.0 | 1 | 15 | 6 | 8 | 130.0 | 65 |
| 3 | 2499.5 | 1 | 0 | 3 | <NA> | -12.5 | 90 |
| 4 | 5499.0 | 1 | 20 | 5 | 21 | 80.0 | 120 |
| 5 | 0.0 | 1 | 0 | 4 | 35 | 45.0 | 0 |
| 6 | 1899.0 | <NA> | <NA> | 4 | 17 | 18.0 | 45 |
| 7 | 3200.0 | -1 | 5 | 0 | 6 | -150.0 | <NA> |
| 8 | 1799.99 | 1 | 125 | 99 | -4 | <NA> | 65 |

Filling missing values:



```
for col in numeric_cols:
    df[col] = df[col].fillna(df[col].median())

categorical_cols = [
    "city", "category", "item_condition", "sales_channel",
    "payment_method", "order_status", "authenticity_check",
    "seller_tier", "buyer_type", "return_flag"
]

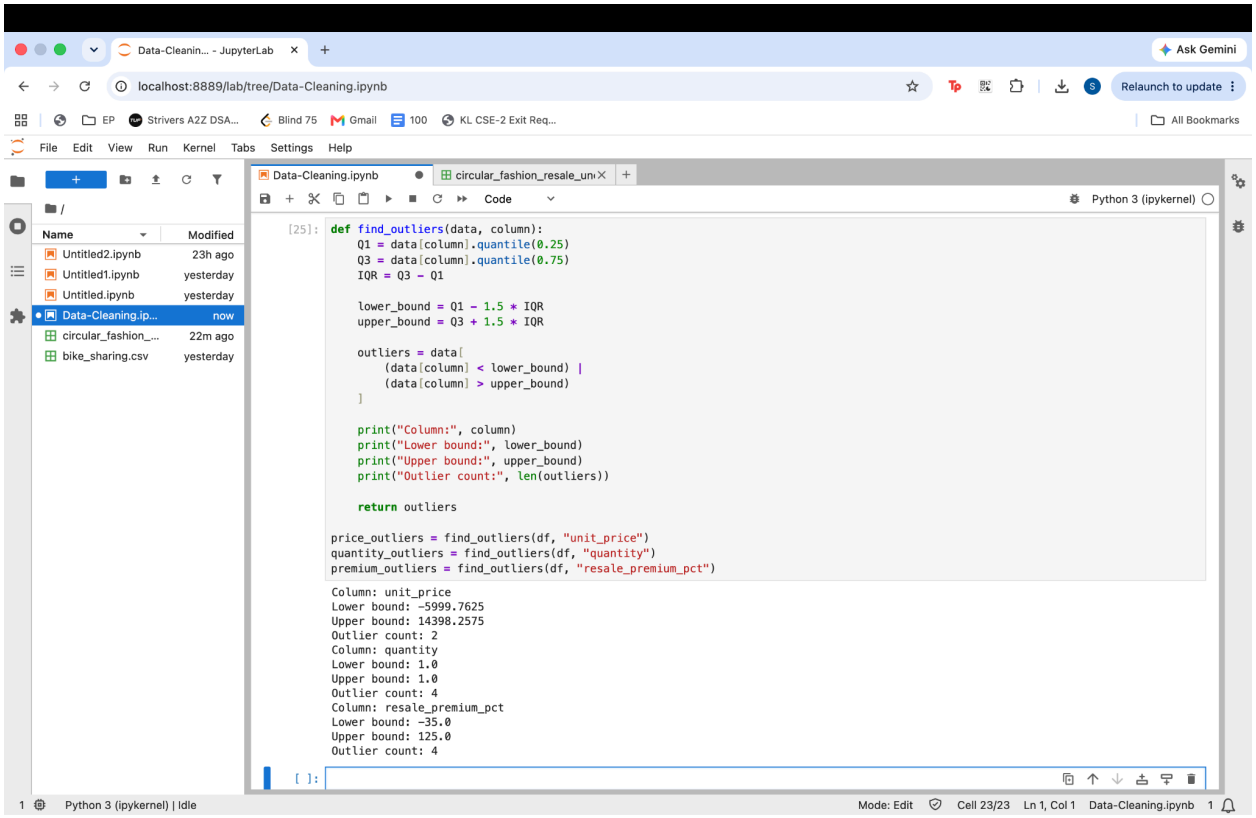
for col in categorical_cols:
    if df[col].notna().any():
        df[col] = df[col].fillna(df[col].mode()[0])

df["buyer_id"] = df["buyer_id"].fillna("Unknown")
df["seller_id"] = df["seller_id"].fillna("Unknown")

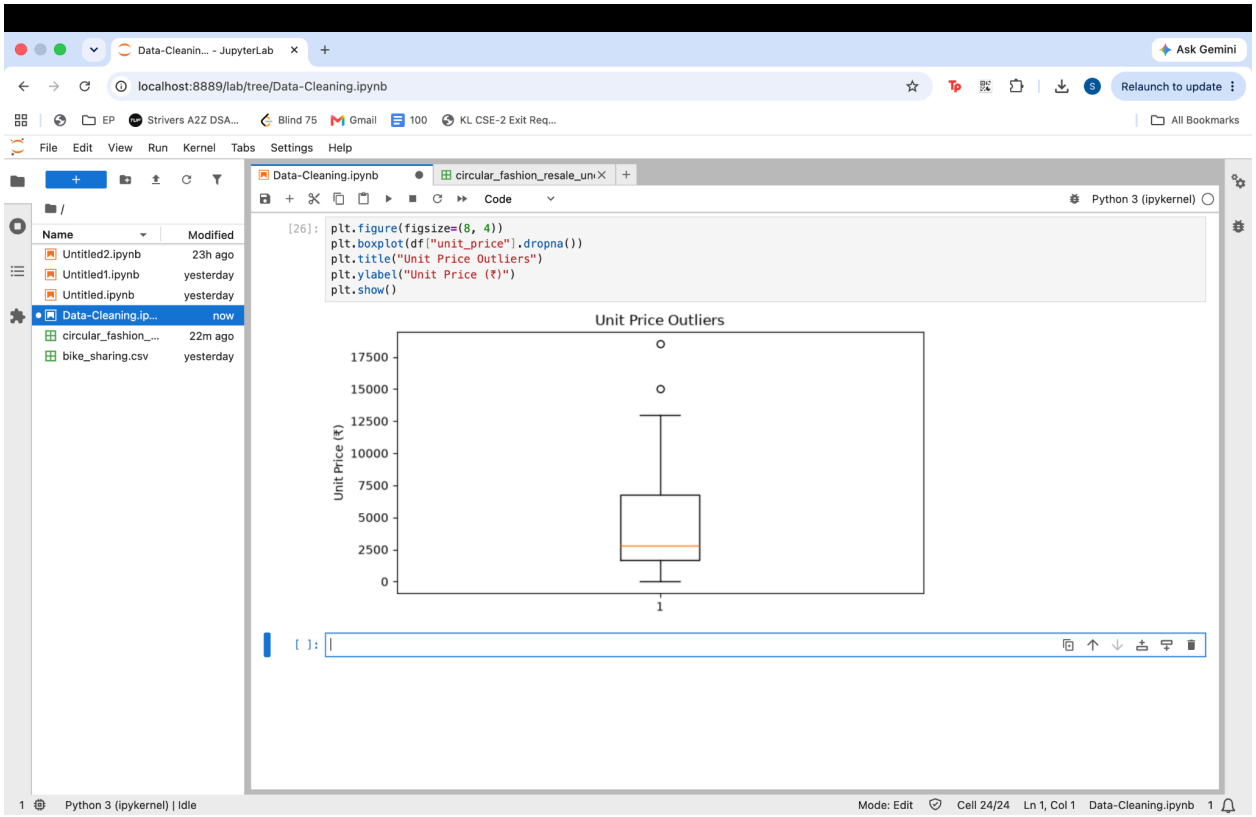
df.isnull().sum()
```

```
listing_id                0
order_date                3
buyer_id                  0
seller_id                 0
item_name                 0
category                  0
item_condition            0
unit_price                0
quantity                  0
discount_pct              0
city                      0
sales_channel             0
payment_method            0
order_status              0
authenticity_check        0
seller_rating             0
days_listed_before_sale   0
resale_premium_pct        0
seller_tier               0
buyer_type                0
```

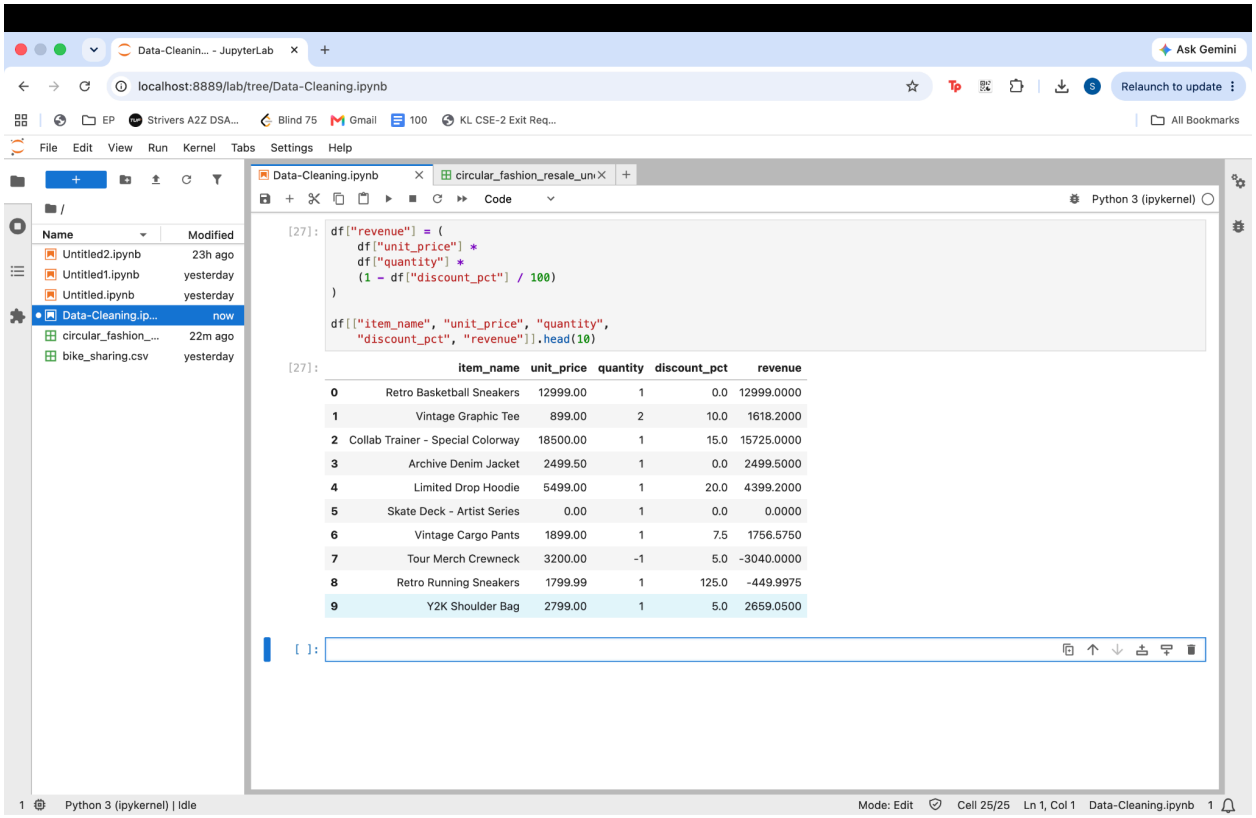
Finding outliers:




Visualizing unitprice outliers:



Creating revenue column:




Categorizing prices:

